

동V리 논고 m장영의문적 아선점차조 아열살반점 재선점실 점열 점열중을
살창점살창점장무열알0말세

Il 7n5g 7y 검m5y 7y 검m5검m5o 기 논결기
lr 낮개xnaVc 근갈능는문: A5근기H5개7: BBA략BDD5?9?9
lxgr k 과는검g 단평개평13gr 검m5과문과26 Tpp

[illegible]

SV;H m7H

1

KD-트리 (축 정렬 분할):

- 고차원에서 성능 저하 (차원의 저주)
- 각도 정보 활용 부족

LSH (확률적 해싱):

- 높은 거짓양성율
- 파라미터 조정 복잡

HNSW (그래프 기반):

- 높은 메모리 오버헤드
- 삽입/삭제 비효율적

VHP의 목표:

- 고차원에서의 효율적 분할
- 메모리 효율성
- 삽입/삭제 지원

SV:N

1단계양해공통형

2D: 원 (circle)
 $\{p \mid d(p, c) = r\}$

3D: $\exists$ (sphere)
 $\{p \mid ||p - c|| = r\}$

고차원: 초구 (hypersphere)
 $\{p \mid ||p - c|| = r\}$

1xqr 2

개념:
하나의 중심점과 여러 반지름으로 초구들을 생성
→ 동심원 구조 (concentric spheres)

예시:
중심: c
초구 1: 반지름 r_1 내의 점들
초구 2: r_1 < 거리 <= r_2 영역의 점들
초구 3: r_2 < 거리 <= r_3 영역의 점들
...

장점:
- 중심에 가까운 점일수록 더 세밀한 분할
- 각도 정보와 거리 정보 동시 활용

SV;P 논문

xgr |

루트 중심: 전체 데이터의 중심
↓
중심에 가장 가까운 N_1개:
- 중심점으로 지정
- 각 중심별로 초구 분할
↓
각 초구별로 다시 분할:
- 부분 중심점 선택
- 해당 영역을 초구로 분할
↓
리프 노드: 실제 벡터들

|

단계 1: 전역 중심 선택
- 데이터의 무게 중심(centroid)
- 또는 대표점(representative point)

단계 2: 각 계층에서 중심점 선택
- 이전 중심으로부터 거리 기준 선택
- 상호 거리 최대화 (분산 최대화)

단계 3: 거리 기반 분할
- 각 중심별로 가까운 점들을 그룹화
- 그룹 내에서 다시 중심 선택

SV;S 논문

xgr |

```
procedure BUILD_VHP_INDEX(vectors, max_partition_size):
    root = create_node()

    function build_recursive(node, points, depth):
        if len(points) <= max_partition_size:
            // 리프 노드: 벡터들 직접 저장
            node.vectors = points
            return

        // 내부 노드: 중심 선택 및 분할
        centroid = compute_centroid(points)
        node.centroid = centroid

        // 각 점과 중심의 거리 계산
        distances = [distance(p, centroid) for p in points]

        // 거리로 정렬
        sorted_indices = argsort(distances)
```

```

// 여러 그룹으로 분할
num_children = ceil(len(points) / max_partition_size)
group_size = ceil(len(points) / num_children)

for group_idx in range(num_children):
    start = group_idx * group_size
    end = min((group_idx + 1) * group_size, len(points))

    child = create_node()
    child.parent = node
    node.children.append(child)

    // 재귀적으로 자식 노드 구성
    build_recursive(child, points[start:end], depth + 1)

build_recursive(root, vectors, 0)
return root

```

|

```

procedure VHP_SEARCH(query_q, K, node=root):
    candidates = []

    // BFS 또는 DFS로 트리 탐색
    queue = [root]
    visited = set()

    while queue is not empty:
        current = queue.pop(0)

        // 현재 노드의 중심과 쿼리 거리
        centroid_dist = distance(q, current.centroid)

        // 가지치기: 이 노드가 결과를 포함할 수 없다면 스킵
        if should_prune(current, centroid_dist, candidates):
            continue

        // 리프 노드: 모든 벡터와 거리 계산
        if is_leaf(current):
            for v in current.vectors:
                dist = distance(q, v)
                candidates.append((v, dist))

        // 내부 노드: 자식 노드 방문
        else:
            for child in current.children:
                if child not in visited:
                    visited.add(child)
                    queue.append(child)

    // 거리로 정렬하여 가장 가까운 K개 반환
    return sorted(candidates, key=lambda x: x[1])[:K]

```

SV;V **4점준인일사**

|

현재까지 찾은 가장 먼 이웃까지의 거리: d_max

노드의 중심: c

쿼리: q

중심까지의 거리: d_c

가지치기 조건:

노드 내 모든 점이 d_max보다 멀다면

해당 노드는 방문하지 않음

수학적:

$d_c - radius \geq d_{max}$

→ 노드 내 어떤 점도 K개 결과에 포함될 수 없음

|

두 벡터의 각도 관계 활용:

벡터 q, p, c에 대해:

$$\theta = \text{angle}(q - c, p - c)$$

코사인 법칙:

$$d(q, p)^2 = d_q^2 + d_p^2 - 2 \cdot d_q \cdot d_p \cdot \cos(\theta)$$

가지치기:

예상 최소 거리 > d_{max} 이면 스킵

|

검색 진행에 따라 d_{max} 가 줄어듦:

- 가지치기 범위 확대
- 검색 시간 단축

적응형 검색:

처음: 넓은 범위 탐색

중간: 좋은 후보 찾으면서 범위 축소

말미: 좁은 범위로 정교한 검색

SV;W

|

균형 잡힌 분할:

- 각 레벨의 노드 수가 유사
- 최대 깊이 $O(\log n)$
- 검색 시간 $O(\log n)$

불균형 분할:

- 큰 클러스터는 깊게 분할
- 작은 클러스터는 얇게 분할
- 데이터 분포 반영

|

`max_partition_size`의 영향:

작은 크기 (10~100):

- 깊은 트리
- 리프 노드 많음
- 가지치기 효과 좋음
- 메모리 사용 증가

큰 크기 (1000~10000):

- 얇은 트리
- 리프 노드 적음
- 리프 내 선형 탐색 증가
- 메모리 절약

최적값: 보통 100~500

SV;[

|

벡터 데이터: $N * d * 4$ 바이트

중심점 저장: 노드_수 * $d * 4$ 바이트

트리 구조: 노드_수 * (포인터 + 메타데이터)

예 (1M 벡터, 1000차원):

벡터: $1M * 1000 * 4 = 4GB$

중심: 약 $10K * 1000 * 4 = 40MB$

트리: 약 $10K * 100 = 1MB$

총: 약 4.05GB (벡터 대비 1% 오버헤드)

vs HNSW: $4 + 4 * 1M/16 = 4.25GB$ (약 6% 오버헤드)

vs NSG: 유사 수준

SV:] 논고점수 가능 잠재력

	xgr	gpuy	pue
대용량 데이터 처리			
8			

SV;a

6 xgrl

VHP 구성 시 필터 조건 고려:

방법 1: 필터별 별도 VHP

- 각 필터 그룹별로 독립적 VHP
- 높은 메모리 비용
- 매우 빠른 검색

방법 2: 통합 VHP + 필터 검사

- 하나의 VHP 사용
- 트리 탐색 시 필터 조건 확인
- 필터 미충족 노드 스킵
- 메모리 효율적

6 |

필터 조건을 고려한 가지치기:

노드 내 모든 벡터가 필터를 만족하지 않으면:

- 해당 노드 전체 스킵

부분적으로 만족:

- 리프 노드까지 내려가서 개별 확인

이를 통해:

- 필터 선택도 높음: 검색 범위 감소
- 필터 선택도 낮음: 광범위 탐색 필요

SV;HG

1. 근접 이웃

새로운 벡터 x를 인덱스에 추가:

- 루트부터 시작하여 리프 노드 찾기
 - 각 레벨에서 가장 가까운 자식 선택
- 리프 노드 도달:
 - 벡터를 리프에 추가
 - 리프 크기가 max_partition_size 초과?
 - 리프 분할 (split)
- 분할 후:
 - 부모 노드의 중심점 재계산
 - 필요시 상향식으로 재균형

시간 복잡도: $O(\log n)$

1. 배경

벡터 x 를 제거:

1. 벡터가 있는 리프 노드 찾기
2. 벡터 제거:
 - 리프에서 직접 제거
 - 리프가 비어있으면 병합(merge) 고려
3. 재균형:
 - 중심점 재계산
 - 크기 요구사항 확인

시간 복잡도: $O(\log n)$

SV;HH 4 5
1 o u h v 5: ? G 2

구성 시간:

- VHP: 20초
- HNSW: 300초
- NSG: 30초
- VHP가 가장 빠름

메모리:

- VHP: 4.0GB
- HNSW: 4.2GB
- NSG: 4.05GB

검색 속도 (K=10):

- VHP: 1.2ms
- HNSW: 0.8ms
- NSG: 1.1ms

정확도:

- VHP: 96%
- HNSW: 97%
- NSG: 96%

SV;HN 논고

|

1. 빠른 구성
 - 트리 구축이 간단
 - 병렬화 가능
2. 메모리 효율성
 - HNSW보다 적은 오버헤드
 - 공간 분할 구조의 이점
3. 동적 작업 지원
 - 삽입/삭제 $O(\log n)$
 - 트리 재구성 필요 없음
4. 파라미터 단순성
 - max_partition_size만 주요 파라미터
 - 자동 조정 가능

|

1. 검색 속도
 - HNSW보다 약 30~50% 느림
 - 그래프 기반의 효율성 미흡

2. 고차원에서의 변동성
 - 매우 고차원(10K+)에서 성능 저하
 - 공간 분할의 효율성 감소
3. 클러스터링 의존성
 - 데이터가 잘 클러스터되지 않으면 성능 저하
 - 균일 분포에 덜 유리

SV;HP 논고

I

VHP의 계층적 구조는 필터링과 잘 맞음:

- 상위 레벨에서:
- 전체 부분집합의 필터 조건 만족도 파악
 - 분명히 필터 미충족인 노드 조기 제거

- 중간 레벨에서:
- 부분적 일치하는 노드 처리
 - 선택적 하강

- 리프 레벨에서:
- 최종 필터 검사
 - 거리 계산

이를 통해 필터 선택도에 따른 성능 최적화

: 7		I				P			
		P							
? 7		I			5	xgr			P
A 7		xgr I		5xgr	8		P		
B 7	xgr	I		xgr				P	
D 7		I		xgr		P			
E 7	I			5xgr		P			
F 7		I xgr		5		5		P	
G 7		I		5xgr		P			
	P								
H 7		I			5xgr				P
: 97		I xgr		gpuy		P 5			5
	gpuy		P						